

PYTHON — 1 PAGE INTERVIEW CHEAT SHEET

1. Data Types

List — ordered, mutable: [1, 2, 3] **Tuple** — ordered, immutable: (1, 2) **Set** — unique values: {1, 2} **Dict** — key/value: {'id': 1} **String** — immutable text.

2. Functions

`def add(a, b=0): return a+b` ***args** collects positional arguments; ****kwargs** collects keyword arguments. **Lambda**: small anonymous function, e.g. `lambda x: x*2`.

3. OOP

Class = blueprint; **Object** = instance. **Encapsulation** = bundle/protect data; **Inheritance** = reuse parent behavior; **Polymorphism** = same interface, different behavior; **Abstraction** = hide implementation details.

4. Decorators

A decorator modifies/enhances a function without changing its core code. Common pattern: `@decorator`. Useful for logging, authentication, timing, etc.

5. Generators & Iterators

Iterator supports `__iter__()` and `__next__()`. **Generator** uses `yield` and produces values lazily, saving memory. Example: `def nums(): yield 1`.

6. Exception Handling

`try` → risky code, `except` → handle error, `else` → runs if no error, `finally` → always runs. Use specific exceptions instead of a bare `except`.

7. Mutable vs Immutable

Mutable: list, dict, set — can change after creation. **Immutable**: int, float, str, tuple, bool — cannot be changed in place.

8. Comprehensions

Compact collection creation: `[x*x for x in nums if x>0]`. Also available for dicts and sets.

9. Sync vs Async

Sync executes sequentially and waits. **Async** can pause at `await` while other I/O work progresses. In FastAPI, use `async def` mainly for non-blocking I/O.

10. Threading vs Multiprocessing

Threading: useful for I/O-bound work; threads share process memory. **Multiprocessing**: separate processes; useful for CPU-bound work and avoids Python GIL limitations for parallel CPU execution.

11. Common Interview One-Liners

Question	Short answer
List vs Tuple	List mutable; tuple immutable.
is vs ==	<code>is</code> checks identity; <code>==</code> checks value/equality.
Shallow vs Deep Copy	Shallow copies outer object; deep copy recursively copies nested objects.
GIL	CPython lock allowing one thread to execute Python bytecode at a time; I/O threads can still be useful.
PEP 8	Python style guide for readable, consistent code.
Virtual environment	Isolates project dependencies from the system Python.

FastAPI reminder: Pydantic validates request/response data; `Depends()` injects dependencies; JWT is commonly used for stateless authentication.